

Generalized Structural Skeletons for GBMCCQSD

一般化 $g L[B] = R_{\text{self}} + R_{\text{context}}$ に対応する骨格候補の整理

Preprint-style conceptual report | 2026-09-25

要旨

前段の分野横断検証では、GBMCCQSD の右辺を self と context に分ける構造は多くの代表方程式と整合した一方、左辺の DB/Dt は ODE/PDE 延積分記憶確率離散ハイブリッド系を同一に扱うにはいことが示唆された。本稿は、一般化形 $g L_k[B] = R_{\text{self}} + R_{\text{context}} + R_{\text{unresolved}}$ を採用した場合に、どのような数理・計算骨格が受け皿になり得るかを整理する。結論として、単一の方程式骨格を選ぶより、(i) B の境界、(ii) evolution operator L_k 、(iii) self/context 分解、(iv) unresolved 項、(v) I/O operator 群、の 5 要素を固定インターフェースとして持つ多層骨格が最もよい。

1. 出発点

現行 GBMCCQSD では B を Stack/Queue を持つ Buffer、Q を B へ方向を与える量、G/M/C1/C2/S/D を B を読み書きする I/O として解釈している。分野横断検証から、 $R_{\text{self}}/R_{\text{env}}$ の区別は B の境界選択に依存し、環境・過去履歴・network coupling・外力・場を一括して扱うには env より context の語が一般的であると考えられた。

$$g L_k[B](t) = R_{\text{self}}[B_t] + R_{\text{context}}[B_t, X_{i,t}] + R_{\text{unresolved}}(t)$$

$$X_{i,t} = \{\text{external variables, history, other agents, fields, inputs}\}$$

Candidate structural skeletons compatible with generalized GBMCCQSD

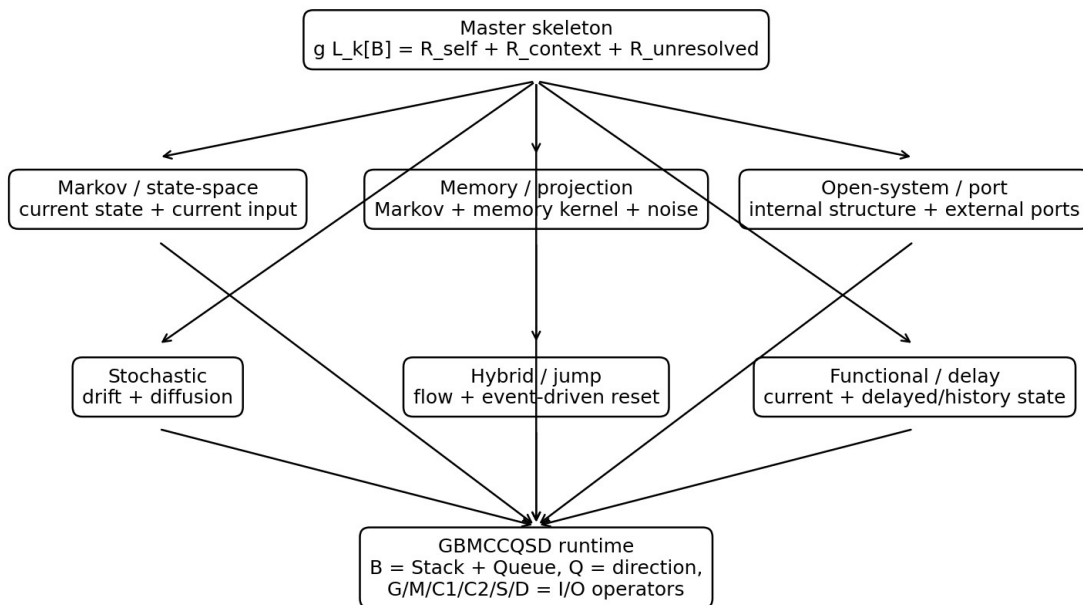


図 1. 一般化 GBMCCQSD を中心にした骨格候補の階層。

2. 骨格 0: Abstract Evolution Operator

最も上位に置けるのは、具体的な微分形式を固定せず作用素 L_k だけを切り替える骨格である。抽象発展方程式では、未知状態が Banach/Hilbert 空間などにし、時間展を作用素で表す。

$$g L_k[B] = R_{\text{self}}[B] + R_{\text{context}}[B, X_i] + R_{\text{unresolved}}$$

$$k \text{ in } \{\text{ODE, PDE, DDE, Volterra, discrete, stochastic, hybrid, ...}\}$$

この骨格は論文上の最上位定義として強いが、抽象度が高いため単独では推定法が決まらない。したがって下位に具体的な発展エンジンを置く必要がある。

3. 骨格 1: Markov State-Space / I-O

$$dB/dt = F_{\text{self}}(B_t, Q_h) + F_{\text{context}}(B_t, X_i_t, Q_c)$$

$$Y_t = H(B_t, Q_t)$$

現在状態と現在入力から次状態を生成する標準的な状態空間骨格。B を state、G/M/C1/C2/S/D を I/O、Q を direction/control として置きやすく、GBMCCQSD の実装基準として最も簡潔である。ただし過去依存を現在 B へ十分に埋め込めない場合は不足する。

4. 骨格 2: Mori-Zwanzig / Generalized Langevin

$$dB/dt = F_M(B_t) + \int_0^t K(t-\tau) \Gamma(B_\tau) d\tau + \eta_t$$

高次元系のうち B だけを resolved variable として保持し、B 外へ落とした自由度の影響を Markov 項、memory kernel、fluctuating force として戻す骨格。これは『B の境界を決めると self/context が決まる』という考えと直接する。Stack を memory、Queue を未処理・将来候補、 η_t を未解像自由度の寄与として対応づける余地がある。

5. 骨格 3: Functional / Delay / Volterra

$$dB(t)/dt = F(B_t, B_{(t-\tau)}, X_i_t)$$

$$dB(t)/dt = F(B_t, \int_0^t K(t,\tau)B(\tau)d\tau, X_i_t)$$

過去を B へ完全圧縮せず、履歴そのものを引数として残す骨格。functional-differential equation は遅延や連続的 after-effect を扱え、Volterra 型は過去間にわたる記憶を持つ。Q_h が現在状態だけでなく履歴窓に依存するなら候補になる。

6. 骨格 4: Stochastic

$$dB_t = [R_{\text{self}}(B_t) + R_{\text{context}}(B_t, X_i_t)]dt + \text{Sigma}(B_t, X_i_t)dW_t$$

epsilon_tを単なる誤差でなく生成機構として扱う骨格。SDE では drift と diffusion が別物である。状態依存で揺らぎの大きさが変わる場合、R_unresolved を一個の加法残差に潰さず Sigma dW として分離する。

7. 骨格 5: Hybrid / Event-Driven

$$\text{flow: } L_k[B] = R_{\text{self}}^q + R_{\text{context}}^q$$

$$\text{jump: } B_{\text{plus}} = J_q(B, X_i)$$

連展とイベントによる不連更新を分ける骨格。GBMCCQSD の『必要な処理だけその場で生成する』思想と相性が良い。Queue が空なら G、試行が必要なら M、確定なら C2/Q、失敗なら Stack から rollback、という処理は固定 8 段パイプラインより event-driven state machine として自然である。

8. 骨格 6: Port-Hamiltonian / Open-System

$$dB/dt = [J(B) - R(B)] \text{grad } H(B) + G(B) u_{\text{context}}$$

$$y = G(B)^T \text{grad } H(B)$$

保存量・貯蔵量 H を定義できる場合、内部構造と外部交換を明示的に分離できる骨格。内部 interconnection/dissipation が self、port を通る入力 u が context に対応する。ただし GBMCCQSD の『責任』『候補』に H のような貯蔵量が本当に存在するかは未確定なので、普遍骨格というより特殊化である。

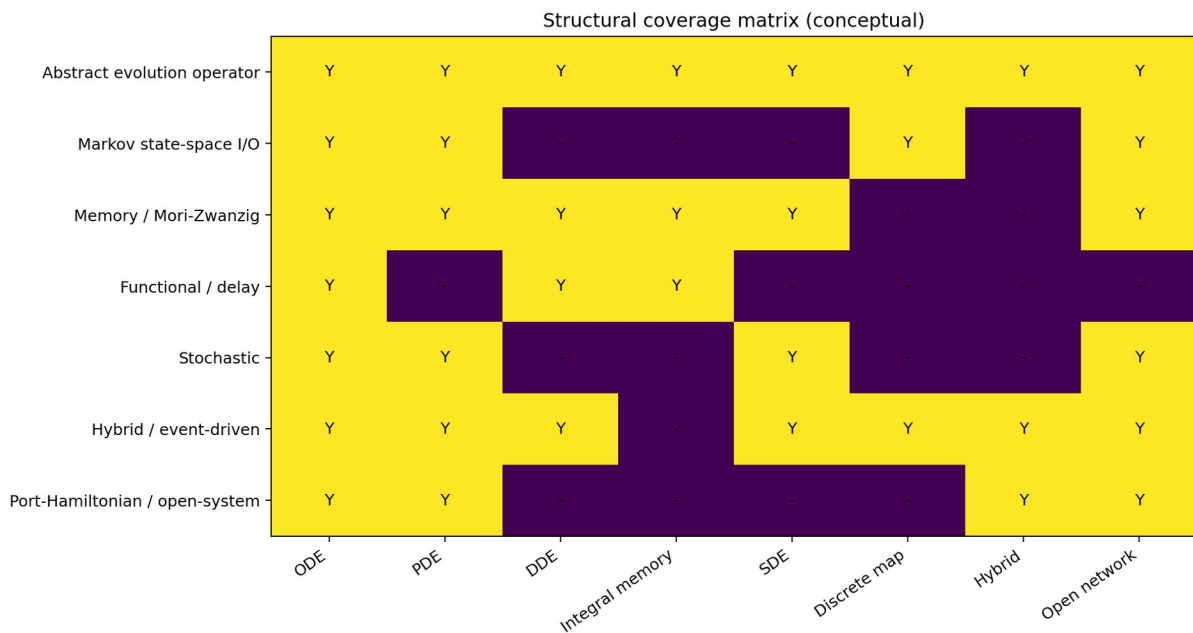


図 2. 各骨格が扱いやすい方程式クラスの念的。

9. 最も相性が良い二層骨格

GBMCCQSD では、意味論・データ構造と方程式クラスを分離する二層構造が最も扱いやすい。

$$\text{Layer A: State}=(B,Q), B=(\text{Queue},\text{Stack}), I/O=\{G,M,C1,C2,S,D\}$$

$$\text{Layer B: } g L_k[B] = R_{\text{self}} + R_{\text{context}} + U_k$$

Layer A を GBMCCQSD 固有の骨格として固定し、Layer B のみを ODE/DDE/Volterra/SDE/hybrid 等へ差し替える。これにより『GBMCCQSD とは何か』を保ったまま、実データが支持する数理クラスだけを交換できる。

10. Q_h / Q_c の配置規則

現在案の $Q_h \rightarrow \text{self}$ 、 $Q_c \rightarrow \text{context}$ は、名前ではなく B の境界で判定する方が一般化しやすい。習慣履歴が Stack 内に明示保持され、現在 B から評価できるなら Q_h は self。履歴を B 外の memory kernel として扱うなら同じ habit 情報でも context/memory 側へ移る。したがって分類規則は『その時点の B 内部だけから計算可能か』とするのが一貫的である。

11. 推する最終メタ骨格

$$\text{State: } X_t = (B_t, Q_t), B_t = (\text{Queue}_t, \text{Stack}_t)$$

$$\text{Boundary: } \Omega_B(t) = \text{variables explicitly represented inside B}$$

$$\text{Context: } X_{i,t} = \text{variables/history/agents/fields outside } \Omega_B(t)$$

$$\text{Evolution: } g L_k[B] = R_{\text{self}}[B, Q; \Omega_B] + R_{\text{context}}[B, Q, X_i; \Omega_B] + U_k$$

$$\text{Runtime: } X_{t+1} = O_t(X_t), O_t \in \{G, M, C1, C2, S, D, \text{rollback}\}$$

U_k は方程式クラス固有の項で、ODE なら 0、Volterra なら memory、SDE なら Sigma dW、hybrid なら jump operator 等を持つ。これで self/context という意味論と数理クラスを分離できる。

12. 検証プロトコル

1. 前に B 境界 Ω_B を固定する。
2. 同一データに Markov, delay, Volterra, stochastic, hybrid の L_k を競合させる。
3. $R_{\text{self}}/R_{\text{context}}$ あり・なし、 Q_h/Q_c 分離あり・なしを factorial 比較する。
4. 未来 holdout だけでなく未見 agent・未見 environment・未見 task へ外挿する。
5. 差自己相→memory/delay、状態依存分散→stochastic、regime 依存→hybrid を検査する。
6. sparse discovery / symbolic regression で手作業立式を反証可能にする。

13. 優先順位

次の実装優先度は、(1) State-space を基準モデル、(2) Mori-Zwanzig/Volterra を memory モデル、(3) Hybrid を動的 I/O モデル、(4) SDE を揺らぎモデルとして並列比較する構成が良い。Port-Hamiltonian は H に相当する貯蔵量・コストが明示できた時点で追加する。

14. 結論

$g L[B] = R_{\text{self}} + R_{\text{context}}$ へ変更しても GBMCCQSD の骨格は失われない。むしろ B/Q/I-O という計算骨格を上位に残し、L を交換可能な発展エンジンにすることで、ODE/PDE だけでなく遅延、記憶、確率、ハイブリッ

ドまで包含できる。最も有望なのは、Abstract Evolution Operator を最上位、State-Space を基準実装、Mori-Zwanzig を境界外自由度、Hybrid を動的処理選択として組み合わせる多層構造である。

参考文献

Encyclopedia of Mathematics. Differential equation, abstract; Evolution equation.

Widder, C., Zimmer, J., & Schilling, T. (2025). On the generalized Langevin equation and the Mori projection operator technique. arXiv:2503.20457.

Widder, C. & Schilling, T. (2026). Generalised Langevin Dynamics: Significance and Limitations of the Projection Operator Formalism. arXiv:2604.20453.

van der Schaft, A. & Jeltsema, D. (2014). Port-Hamiltonian Systems Theory: An Introductory Overview. Foundations and Trends in Systems and Control, 1(2-3), 173-378. DOI: 10.1561/2600000002.

Encyclopedia of Mathematics. Stochastic differential equation.

Encyclopedia of Mathematics. Functional-differential equation; Integro-differential equation; Abstract Volterra equation.

Encyclopedia of Mathematics. Automatic control theory.